UNIVERSIDAD AUTÓNOMA DE NUEVO LEÓN

FACULTAD DE INGENIERÍA MECÁNICA Y ELÉCTRICA

Unidad De Aprendizaje: Sistemas Operativos

Semestre: Agosto- Diciembre 2026

Actividad Fundamental 2: Multitarea Y Control de Concurrencia

Catedrático: Dra. Marín Martínez Norma Edith

Hora: M4 Grupo: 001 Plan: 440 Equipo: 3 Aula: 4206

Paralelismo Fases de concurrencia

Fotografía	Matrícula	Nombre	Carrera	Porcentaje
	2173227	Chávez Romero Abner Daniel	IAS	100%
	2145945	Dávila Gongora Leonardo	ITS	100%
	2094613	Del Ángel Hernández Edgar Daniel	ITS	100%
	2149169	Flores Dávila Andrea Sarahy	IAS	100%

Dra. Norma Edith Martínez Ma.

21 AGO 2026

REVISADO

Escaneado con CamScanner

	2145905	Garza Jasso Erubiel Elías	IAS	100%
	2145422	Marroquín Tobías José Eduardo	IAS	100%
	2052641	Martínez Alvárez Alejandro	ITS	100%
	2153829	Martínez García Donovan Alexander	IAS	100%
	2147723	Molina Garcia Jorge Luis	ITS	100%
	2145757	Morales Quintanilla Alonso	ITS	100%

Escaneado con CamScanner

	2226953	Revuelta Martínez Carolina	IAS	100%
	2149382	Ruiz Treviño Esdras Alexander	IAS	85%
	2144697	Salazar Arzola Karla Nozomi	IAS	100%

A 21 de Agosto del 2026. San Nicolás de los Garza, Nuevo León, México.

Escaneado con CamScanner

UNIVERSIDAD AUTÓNOMA DE NUEVO LEÓN

FACULTAD DE INGENIERÍA MECÁNICA Y ELÉCTRICA

Unidad De Aprendizaje: Sistemas Operativos

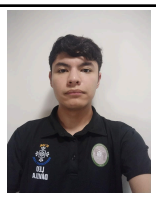
Semestre: Agosto- Diciembre 2026

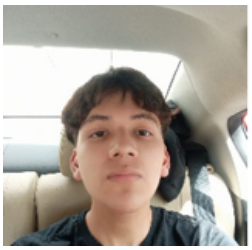
Actividad Fundamental 2: Multitarea Y Control de Concurrencia

Catedrático: Dra. Marín Martínez Norma Edith

Hora: M4 Grupo: 001 Plan: 440 Equipo: 3 Aula: 4206

Enlace del video: https://youtu.be/hQXW7AQzONo

Fotografía	Matrícula	Nombre	Carrera	Porcentaje
	2173227	Chávez Romero Abner Daniel	IAS	100%
	2145945	Dávila Gongora Leonardo	ITS	100%
	2094613	Del Ángel Hernández Edgar Daniel	ITS	100%
	2149169	Flores Dávila Andrea Sarahy	IAS	100%

	2145905	Garza Jasso Erubiel Elías	IAS	100%
	2145422	Marroquín Tobías José Eduardo	IAS	100%
	2052641	Martínez Alvárez Alejandro	ITS	100%
	2153829	Martínez García Donovan Alexander	IAS	100%
	2147723	Molina Garcia Jorge Luis	ITS	100%
	2145757	Morales Quintanilla Alonso	ITS	100%

	2226953	Revuelta Martínez Carolina	IAS	100%
	2149382	Ruiz Treviño Esdras Alexander	IAS	85%
	2144697	Salazar Arzola Karla Nozomi	IAS	100%

A 2121 de Agosto del 2026. San Nicolás de los Garza, Nuevo León, México.

Contenido

Índice de imágenes ..... **¡Error! Marcador no definido.**
Introducción 3
Definición de concurrencia 4
Tipos de concurrencia 5
Modelos de programación concurrente 10
Ventajas de la concurrencia 17
Tipos de Procesos concurrentes 19
Tipos de interacción entre los procesos de la concurrencia 21
Cuadro sinóptico 23
Conclusión General 24
Conclusiones personales 25
Bibliografía 32
Webgrafía 33
Bibliografía de imágenes 34

Ilustración 1.Ejemplo visual del uso del CPU en la concurrencia. 4
Ilustración 2. Formas de trabajar de la concurrencia 5
Ilustración 3. Ejemplo de ejecución de un sistema multitarea 6
Ilustración 4. Ejemplo de pseudoparalelismo 7
Ilustración 5. Multiprocesamiento en SO 9
Ilustración 6. Ejemplo de Concurrencia Real 10
Ilustración 7. Modelo de comunicación interprocesos (IPC) por memoria compartida 12
Ilustración 8. Modelo de paso de mensajes 13
Ilustración 9. Modelo de actores 15
Ilustración 10. Modelo de flujo de datos (DataFlow) 16
Ilustración 11. Concurrencia de un sistema operativo 17
Ilustración 12. Mecanismo de paso de mensajes gestionado a través del núcleo (Kernel) del sistema operativo. 20
Ilustración 13. Comunicación entre procesos concurrentes mediante el uso de una tubería (pipe). 20
Ilustración 14. Ejemplos de tipos de interacción entre los procesos 22

Introducción

La multitarea es una de las funciones fundamentales de los sistemas operativos modernos, ya que permite ejecutar y administrar varios procesos de manera concurrente, aprovechando de forma eficiente los recursos del equipo. Para lograrlo, el sistema operativo debe coordinar el acceso de los diferentes procesos y subprocesos a recursos compartidos como la memoria, el procesador y los archivos, evitando conflictos que puedan provocar errores o comportamientos inesperados.

Dentro de este contexto, el **control de concurrencia** comprende el conjunto de mecanismos y técnicas utilizados para organizar la ejecución simultánea de procesos, garantizando que las operaciones se realicen de manera correcta y segura. Conceptos como procesos concurrentes, exclusión mutua, sincronización, semáforos y regiones críticas son fundamentales para comprender cómo un sistema operativo mantiene la integridad de la información mientras múltiples tareas se ejecutan al mismo tiempo.

En este reporte se abordará el funcionamiento de la multitarea y el control de concurrencia, así como los principales mecanismos empleados por los sistemas operativos para coordinar procesos y evitar problemas como condiciones de carrera, bloqueos y accesos simultáneos incorrectos a los recursos.

Definición de concurrencia

La concurrencia se refiere a la capacidad de un sistema para ejecutar múltiples procesos o hilos de manera que sus intervalos de ejecución se solapen. Esto no implica necesariamente que se ejecuten simultáneamente en el mismo instante, sino que el sistema puede alternar entre ellos, dando la impresión de simultaneidad.

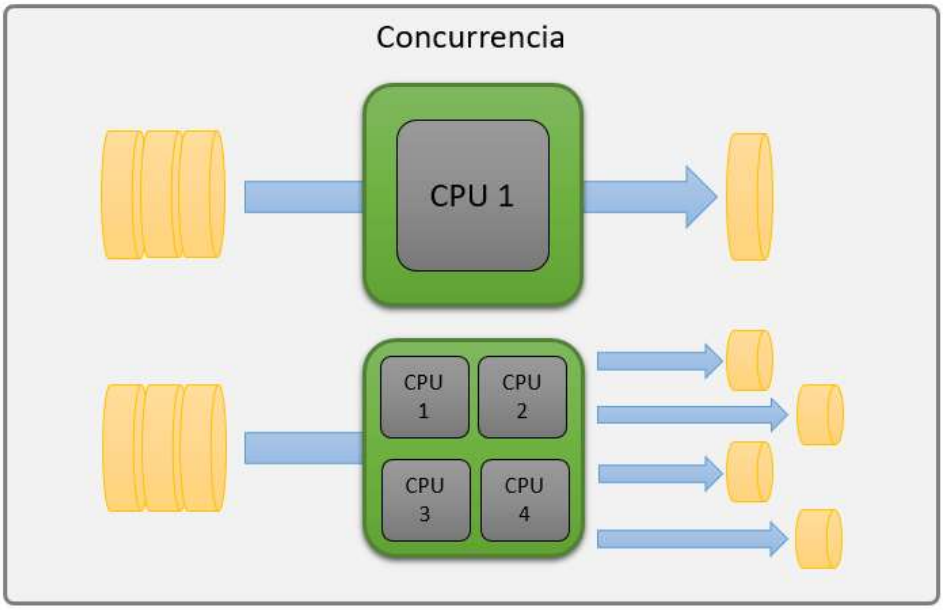


Ilustración 1.Ejemplo visual del uso del CPU en la concurrencia.

Una de las funciones principales de los sistemas operativos es dar a cada proceso la idea de que se está ejecutando en una computadora independiente, de manera que el programador no tenga que preocuparse constantemente por la competencia entre procesos para utilizar los recursos del sistema. Sin embargo, en determinadas situaciones, un programa necesita comunicarse e interactuar con otros procesos o hilos para poder realizar correctamente sus funciones.

Esta interacción puede ser necesaria cuando una parte del procesamiento depende de información obtenida desde fuentes externas. Además, la colaboración entre diferentes hilos o procesos puede ser fundamental para coordinar tareas, compartir información y permitir que determinadas operaciones se lleven a cabo de manera conjunta.

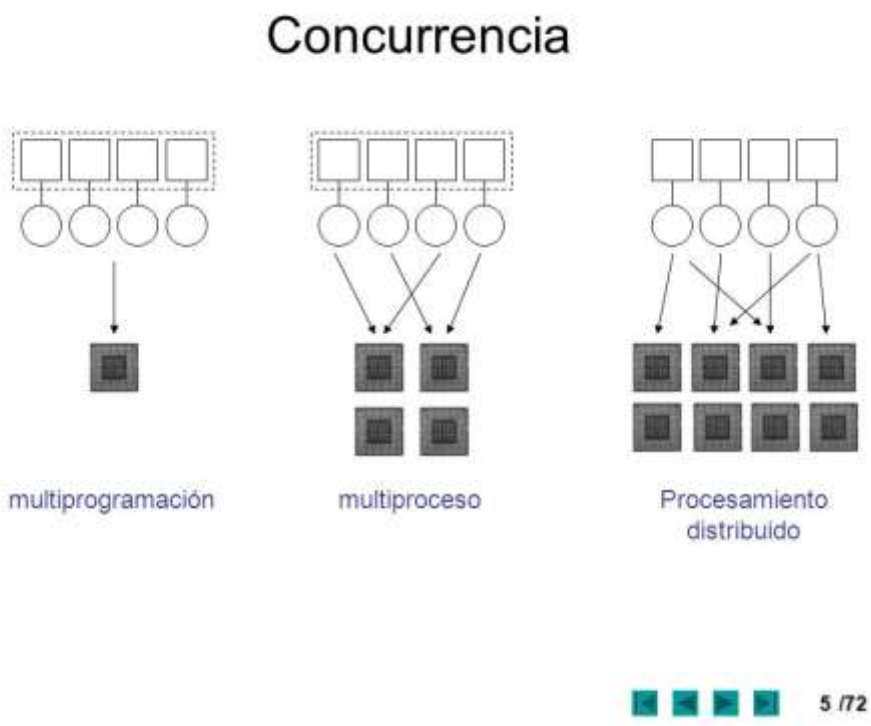


Ilustración 2. Formas de trabajar de la concurrencia

Tipos de concurrencia

Dentro de los sistemas operativos se pueden identificar dos tipos principales de concurrencia: la **concurrencia aparente** y la **concurrencia real**. Ambas permiten que diferentes tareas puedan avanzar de manera concurrente, aunque la forma en que se lleva a cabo su ejecución es diferente. A continuación, se explicará la concurrencia aparente y sus principales características.

Concurrencia aparente

La concurrencia aparente es un tipo de concurrencia que se presenta cuando existen más procesos que procesadores disponibles para ejecutarlos. En este caso, los procesos no se ejecutan verdaderamente de manera simultánea, sino que el sistema operativo intercala su ejecución para permitir que todos puedan avanzar. Debido a la rapidez con la que se realizan estos cambios, para el usuario puede parecer que varios procesos están funcionando al mismo tiempo.

Funcionamiento

En la concurrencia aparente, los procesos se multiplexan en el tiempo, es decir, el procesador distribuye su tiempo de ejecución entre los diferentes procesos. El sistema operativo se encarga de planificar qué proceso utilizará el procesador y durante cuánto tiempo. Una vez que un proceso deja de utilizarlo, otro puede continuar su ejecución, y este procedimiento se repite constantemente.

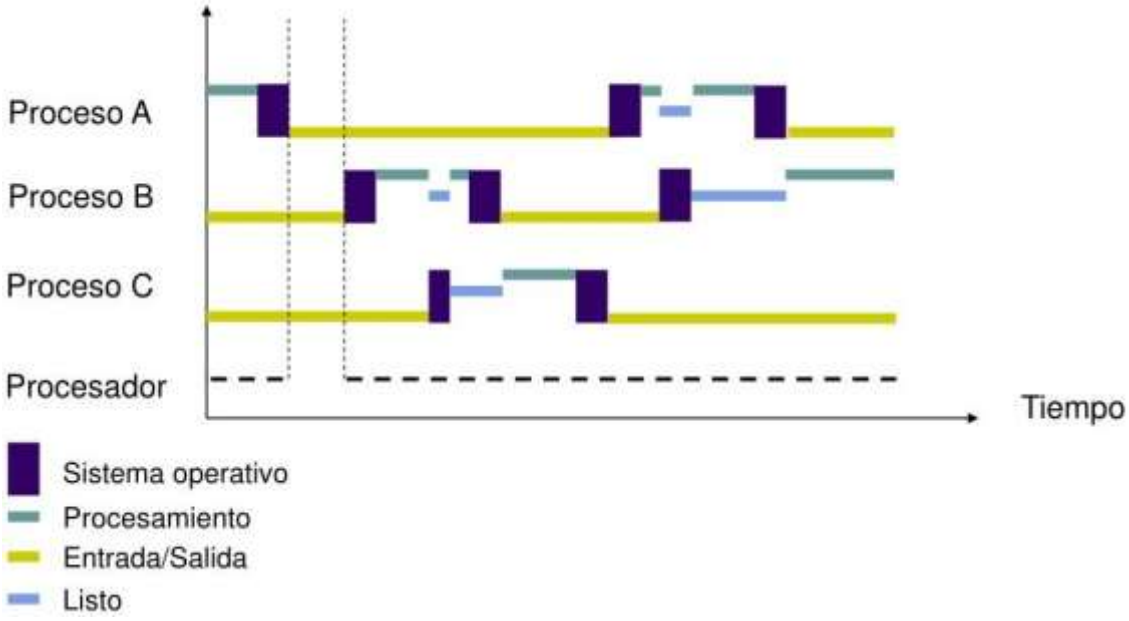


Ilustración 3. Ejemplo de ejecución de un sistema multitarea

Por ejemplo, si una computadora cuenta con un solo procesador y tiene varios procesos activos, el procesador puede ejecutar durante un pequeño intervalo de tiempo una parte del proceso A, después cambiar al proceso B y posteriormente al proceso C. Después puede regresar nuevamente al proceso A. De esta manera, los procesos van avanzando de forma intercalada, aunque en un instante determinado solamente uno esté utilizando el procesador. El libro Sistemas operativos: una visión aplicada utiliza precisamente este modelo para explicar la ejecución concurrente en un sistema con un único procesador.

Pseudoparalelismo

A este comportamiento se le conoce como pseudoparalelismo, debido a que existe una apariencia de ejecución simultánea sin que realmente todos los procesos

se estén ejecutando al mismo tiempo. El usuario percibe que las aplicaciones funcionan de manera simultánea, pero en realidad el procesador está alternando rápidamente entre los diferentes procesos.



Ilustración 4. Ejemplo de pseudoparalelismo

Por ejemplo, un usuario puede tener abierto un navegador, un reproductor de música y un procesador de textos. En un sistema con un único procesador, el sistema operativo puede alternar rápidamente entre las tareas de estas aplicaciones, haciendo posible que todas continúen funcionando y dando la impresión de que se ejecutan al mismo tiempo.

Características principales

Entre las principales características de la concurrencia aparente se encuentran:

- Existe una cantidad de procesos mayor que la cantidad de procesadores disponibles.
- Los procesos se ejecutan de manera intercalada mediante la multiplexación en el tiempo.
- El sistema operativo se encarga de distribuir el tiempo del procesador entre los diferentes procesos.

- Se produce pseudoparalelismo, ya que la ejecución parece simultánea, aunque no lo sea realmente.
- Permite que múltiples procesos permanezcan activos y avancen durante un mismo periodo de tiempo.

Importancia en los sistemas operativos

La concurrencia aparente permite aprovechar de mejor manera el procesador cuando existen varios procesos que necesitan ejecutarse. En lugar de mantener un proceso ejecutándose continuamente hasta que termine, el sistema operativo puede intercalar diferentes procesos y mantener activa la computadora para realizar varias tareas. Este mecanismo es especialmente importante en sistemas de multiprogramación con un único procesador, donde el sistema operativo se encarga de repartir el tiempo entre los procesos mediante la planificación.

Ejemplo

Un ejemplo cotidiano puede observarse cuando una computadora tiene abierto un navegador, un programa para reproducir música y un procesador de textos. Aunque el usuario percibe que las tres aplicaciones funcionan simultáneamente, si solo existe un procesador disponible, el sistema operativo debe repartir el tiempo de procesamiento entre ellas. Así, cada proceso obtiene pequeños intervalos de ejecución y todos pueden continuar avanzando. Esta alternancia rápida genera la apariencia de ejecución simultánea característica de la concurrencia aparente.

Por otro lado, se tiene la concurrencia real:

Concurrencia real

Este tipo de concurrencia se define como aquella concurrencia que existirá cuando exista un proceso por procesador, en este caso se habla de un paralelismo real. Cabe destacar que sólo se cuenta con concurrencia real en sistemas multiprocesador, ya que en un sistema uniprocesador existe una concurrencia aparente.

Características principales

Entre las principales características de la concurrencia real se encuentran:

- La capacidad de ejecución simultánea que contiene el sistema operativo de administrar y llevar a cabo múltiples tareas o procesos.
- El paralelismo real que se da, por ejemplo, cuando se van a ejecutar instrucciones del programa con ayuda de más de un procesador en uso simultáneo.
- El multiprocesamiento de gestionar varios procesos dentro de un multiprocesador.

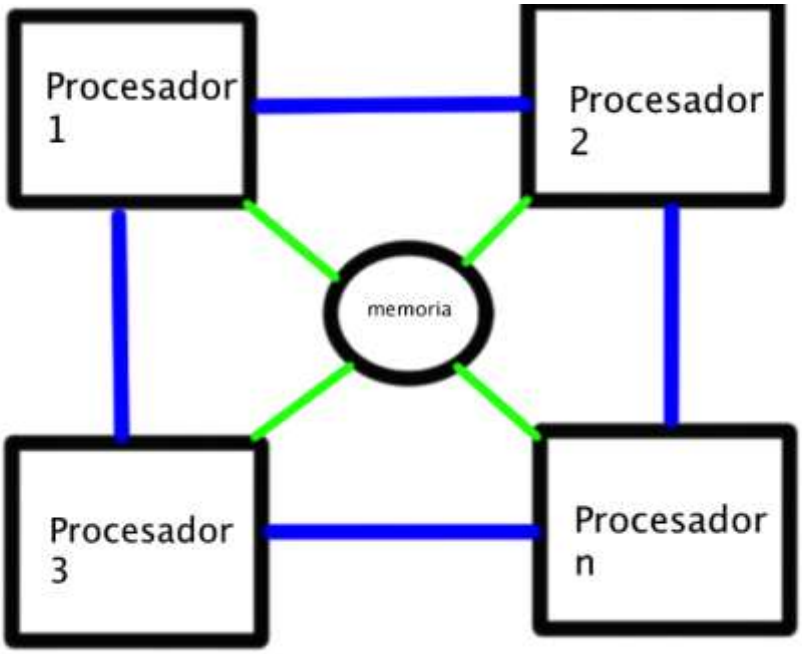


Ilustración 5. Multiprocesamiento en SO

Ejemplo

La concurrencia real ocurre en nuestra vida diaria de manera regular, por ejemplo, cuando estamos escuchando música, mientras descargamos un archivo y enviamos mensajes a la vez. Si nuestro teléfono cuenta con varios procesadores o núcleos, se presenta la oportunidad de que varias tares se ejecuten simultáneamente.



Ilustración 6. Ejemplo de Concurrencia Real

Modelos de programación concurrente

Un modelo de programación concurrente es una estructura conceptual que define la manera en que los programas organizan, coordinan e intercambian información entre múltiples tareas o procesos que se ejecutan de manera simultánea o de forma intercalada. Cuando un sistema operativo o una aplicación necesita ejecutar más de un proceso a la vez, surge la necesidad de establecer reglas claras para que dichas tareas no interfieran negativamente entre sí al acceder a los recursos de la computadora. Los modelos de programación proporcionan precisamente las herramientas lógicas y los patrones de diseño que permiten a los programadores construir software concurrente de manera ordenada, segura y eficiente.

1. Modelo de Memoria Compartida (Shared Memory)

El modelo de memoria compartida es uno de los esquemas más tradicionales y utilizados en la informática, especialmente en sistemas que cuentan con un único procesador dividido en múltiples núcleos o en arquitecturas de multiprocesamiento simétrico (SMP).

- **Funcionamiento básico:** En este modelo, los diferentes procesos o hilos de ejecución comparten un mismo espacio de dirección dentro de la memoria RAM principal. Esto significa que un hilo puede escribir un dato en una variable situada en la memoria e, inmediatamente después, otro hilo diferente puede leer o modificar ese mismo dato. Una analogía sencilla es imaginar un pizarrón en una sala de trabajo compartida: cualquier miembro del equipo puede ver lo que está escrito en el pizarrón y realizar anotaciones sobre él.
- **Mecanismos de control y sincronización:** Dado que todos los procesos tienen acceso directo a las mismas zonas de memoria, existe el riesgo de que dos o más tareas intenten modificar la misma información exactamente al mismo tiempo, lo que generaría errores o corrupción de datos (situación conocida como condición de carrera). Para evitar esto, se emplean herramientas de sincronización que garantizan el acceso ordenado a la información:
    - **Exclusión mutua:** Mecanismo que asegura que solo un proceso a la vez pueda estar ejecutando o modificando una zona crítica de datos.
    - **Semáforos:** Herramientas de señalización tipo contador o bandera que permiten o restringen el paso de los procesos hacia los recursos compartidos.
    - **Monitores:** Estructuras de alto nivel que agrupan las variables compartidas y las operaciones permitidas sobre ellas, regulando el acceso seguro de manera automática.
- **Ventajas y desventajas:** La ventaja principal de este modelo es su gran velocidad de comunicación, ya que los datos no necesitan enviarse ni duplicarse a través de la red o del sistema operativo; simplemente residen en la memoria principal. Su mayor desventaja es la complejidad en la programación, puesto que una sincronización mal diseñada puede provocar

bloqueos mutuos (donde dos procesos quedan esperando indefinidamente por un recurso que posee el otro) o datos inconsistentes.

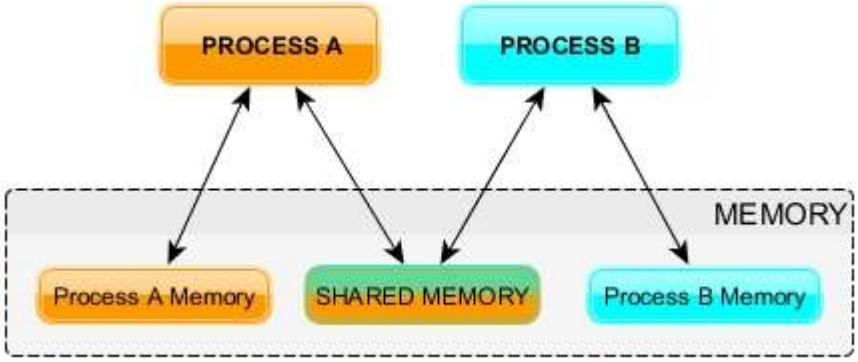


Ilustración 7. Modelo de comunicación interprocesos (IPC) por memoria compartida

2. Modelo de Paso de Mensajes (Message Passing)

El modelo de paso de mensajes surge como una solución estructurada para evitar los peligros de la modificación simultánea de memoria, siendo el pilar fundamental tanto en aplicaciones locales como en sistemas distribuidos, redes de computadoras y esquemas cliente/servidor.

- **Funcionamiento básico:** En este paradigma, los procesos no comparten memoria entre sí. Cada proceso posee su propio espacio de memoria privado e independiente al que ningún otro proceso puede acceder directamente. Para poder cooperar en una tarea, los procesos se comunican enviando y recibiendo bloques de información llamados mensajes. Siguiendo la analogía anterior, en lugar de un pizarrón compartido, cada trabajador está en su propia oficina cerrada y la única manera de enviar información es redactando una carta y enviándola al buzón del compañero.
- **Mecanismos de comunicación:** El intercambio de mensajes suele gestionarse mediante el núcleo (kernel) del sistema operativo o utilizando herramientas como tuberías (*pipes*) y enchufes de red (*sockets*). La comunicación puede presentarse en dos modalidades principales:

- **Comunicación síncrona (bloqueante):** El proceso que envía el mensaje detiene su ejecución hasta que el proceso receptor ha recibido confirmación de la entrega.
- **Comunicación asíncrona (no bloqueante):** El proceso emisor deposita el mensaje en una cola de correo o buzón y continúa ejecutando sus tareas sin quedarse esperando.

- **Ventajas y desventajas:** La principal ventaja de este modelo es la seguridad y el aislamiento: al no existir variables compartidas, es imposible que un proceso corrompa por accidente la memoria de otro. Además, es un modelo altamente escalable que funciona de la misma manera si los procesos están en el mismo equipo o si se encuentran en computadoras distintas conectadas por red. Su principal desventaja es el consumo de tiempo y recursos asociados al empaquetamiento, envío y procesamiento de cada mensaje.

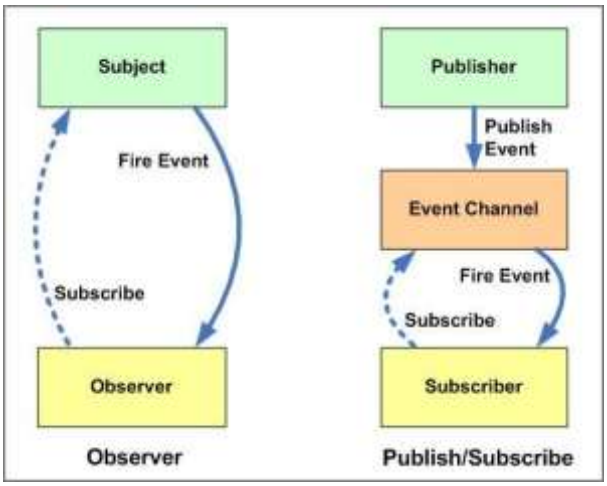


Ilustración 8. Modelo de paso de mensajes

3. Modelo de Actores (Actor Model)

El modelo de actores es un modelo de programación concurrente que lleva el concepto del paso de mensajes a un grado más alto de abstracción y organización modular.

- **Funcionamiento básico:** La unidad fundamental de ejecución en este esquema es el "actor". Un actor es una entidad autónoma e independiente que encierra en su interior tres elementos fundamentales: un estado o datos privados, un comportamiento y una cola de mensajes de entrada (buzón de correo). Los actores no comparten su estado interno bajo ninguna circunstancia.

- **Características principales:**

  - **Aislamiento total:** Los datos contenidos dentro de un actor solo pueden ser leídos o modificados por ese mismo actor.

  - **Procesamiento en cola:** Los mensajes recibidos en el buzón del actor se procesan de forma secuencial, uno por uno.
  - **Acciones simples:** Al procesar un mensaje, un actor únicamente puede realizar tres tareas: enviar un número finito de mensajes a otros actores que conozca, crear nuevos actores independientes o modificar su propio comportamiento interno para responder al siguiente mensaje que reciba.
  - **Alta tolerancia a fallos:** Como cada actor es un módulo aislado, si un actor sufre un error y falla, este incidente no afecta ni interrumpe la ejecución del resto del sistema, lo que facilita la construcción de programas resistentes.

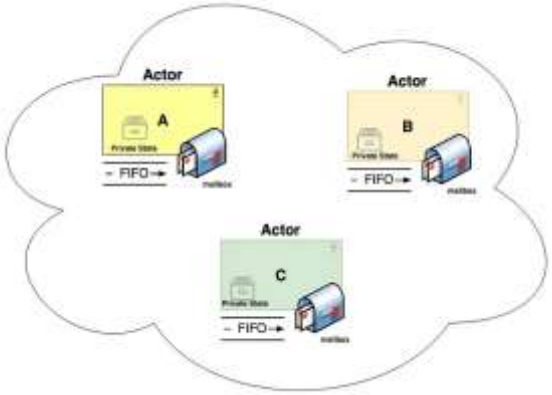


Ilustración 9. Modelo de actores

4. Modelo de Flujo de Datos (DataFlow)

El modelo de flujo de datos aborda la concurrencia desde una perspectiva completamente distinta a la de los modelos tradicionales centrados en dar instrucciones en un orden estricto paso a paso.

- **Funcionamiento básico:** En la programación convencional, el programa dicta de forma explícita qué instrucción debe ejecutarse primero, cuál después y así sucesivamente. En el modelo de flujo de datos, por el contrario, la ejecución de una tarea o cálculo no depende de una secuencia rígida de instrucciones, sino de la disponibilidad de los datos requeridos para realizar la operación.

- **Características principales:**

  - **Ejecución impulsada por datos:** Una función, nodo o bloque de procesamiento se activa y se ejecuta automáticamente en el momento preciso en que todos sus datos de entrada están listos y disponibles.
  - **Ausencia de estado global:** No existen variables globales ni memoria centralizada que guarde estados intermedios. La

información fluye a través de un gráfico o red de operaciones como si se tratara de agua desplazándose por un sistema de tuberías.

- **Paralelismo implícito:** Permite aprovechar de forma natural la capacidad de múltiples procesadores, debido a que todas las operaciones cuyos insumos o datos estén completos pueden ejecutarse simultáneamente sin necesidad de coordinar los tiempos de ejecución de manera manual.

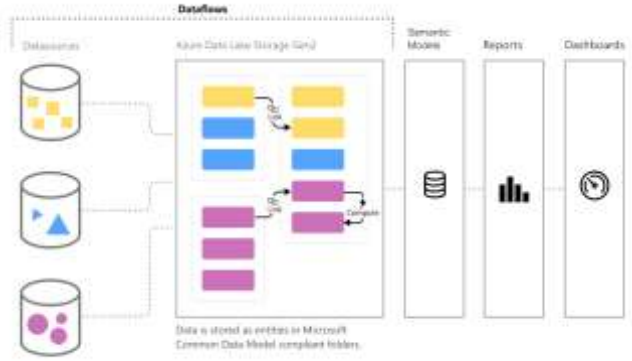


Ilustración 10. Modelo de flujo de datos (DataFlow)

Consideraciones y Comparativa entre Modelos

La elección del modelo de programación concurrente adecuado depende directamente del hardware sobre el cual se ejecutará el software y de la naturaleza del problema a resolver.

Mientras que en el desarrollo de software para un solo equipo con procesador de múltiples núcleos suele preferirse el modelo de memoria compartida por su acceso directo y rápido a la RAM, los entornos modernos de computación en la nube, servicios web masivos y sistemas distribuidos utilizan principalmente el paso de mensajes y el modelo de actores por su seguridad, facilidad de mantenimiento y excelente capacidad para escalar entre múltiples máquinas.

El estudio de estos modelos brinda la base necesaria para comprender cómo los sistemas operativos y las aplicaciones modernas coordinan sus tareas, optimizan el uso del procesador y evitan los conflictos típicos del trabajo multitarea.

Ventajas de la concurrencia

La concurrencia permite que un sistema operativo ejecute múltiples tareas de manera simultánea o intercalada, ofreciendo beneficios clave para el rendimiento y la experiencia del usuario.

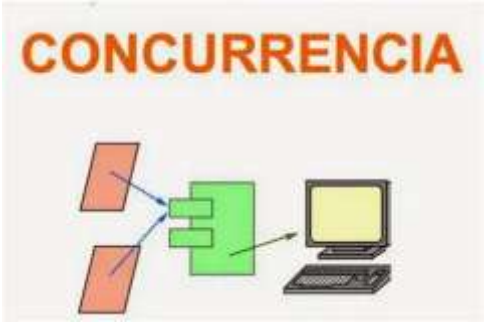


Ilustración 11. Concurrencia de un sistema operativo

1. Mayor capacidad de respuesta

La concurrencia evita que las aplicaciones se bloqueen durante operaciones largas. Mientras un hilo realiza una tarea pesada (como cargar un archivo o descargar datos), otro hilo mantiene la interfaz activa, permitiendo que el usuario siga interactuando con el programa. Esto es esencial en sistemas operativos modernos donde la experiencia del usuario es prioritaria.

2. Mejor aprovechamiento del procesador

Sin concurrencia, la CPU permanece inactiva mientras espera operaciones de entrada/salida (lectura de disco, red, etc.). Con concurrencia, el sistema operativo asigna el procesador a otras tareas durante esos tiempos de espera, manteniendo la CPU ocupada y maximizando el rendimiento. Además, los hilos son más ligeros que los procesos: crearlos y cambiar entre ellos requiere menos tiempo y memoria.

3. Escalabilidad en hardware multinúcleo

En procesadores con múltiples núcleos, diferentes hilos pueden ejecutarse verdaderamente en paralelo, es decir, al mismo tiempo en distintos núcleos. Esto permite dividir tareas complejas en subtareas que se procesan simultáneamente, reduciendo el tiempo total de ejecución. Un programa con un solo hilo solo usa un núcleo; uno con múltiples hilos puede aprovechar toda la potencia del hardware.

4. Comunicación eficiente entre tareas

Los hilos de un mismo proceso comparten memoria y recursos de forma natural, lo que simplifica la comunicación entre ellos. No necesitan mecanismos complejos del sistema operativo (como tuberías o colas de mensajes) para intercambiar información, a diferencia de los procesos independientes. Esto hace que la programación concurrente sea más directa y rápida.

5. Modularidad y simplicidad en el diseño

Muchos problemas del mundo real son inherentemente concurrentes (servidores que atienden múltiples clientes, simulaciones, etc.). Modelarlos con un diseño concurrente es más natural y facilita el desarrollo, mantenimiento y evolución del software. Permite dividir el código en componentes independientes que se comunican entre sí.

6. Tolerancia a fallos

Si un hilo o proceso falla, los demás pueden seguir ejecutándose sin interrupción. Esto hace que el sistema sea más robusto y confiable. En sistemas críticos, permite implementar redundancia y recuperación sin afectar el funcionamiento general.

7. Mayor rendimiento general

Al superponer operaciones de cómputo y entrada/salida, y al distribuir la carga entre múltiples núcleos, se reduce el tiempo de inactividad y se aumenta la cantidad de tareas completadas por unidad de tiempo, mejorando el rendimiento global del sistema.

Tipos de Procesos concurrentes

Tipos de Procesos Concurrentes: Procesos Cooperativos y Comunicación

Dentro del entorno de la multiprogramación, una tarea fundamental de cualquier sistema operativo moderno es la gestión de procesos. Para que el sistema funcione de manera eficiente, el sistema operativo debe reservar recursos para los procesos y permitir a los mismos compartir e intercambiar información. En este contexto, cuando los procesos interactúan entre sí para llevar a cabo una tarea, entran en juego los mecanismos de cooperación y comunicación.

- **Sincronización y Cooperación:** Si los procesos necesitan cooperar, se necesitan algunos métodos de sincronización. El sistema operativo debe proporcionar herramientas para coordinar estas actividades y establecer un control sistemático de su ejecución. Una de las técnicas más sencillas para la sincronización de procesos, y un concepto importante en el diseño de los sistemas operativos, es el semáforo.

- **Comunicación e Intercambio de Mensajes:** A nivel de la estructura del sistema operativo, existen niveles de diseño (como el Nivel 8 en la jerarquía del sistema) que tratan específicamente con la comunicación de información y mensajes entre los procesos. Esta capa permite una compartición de información más rica entre las tareas.

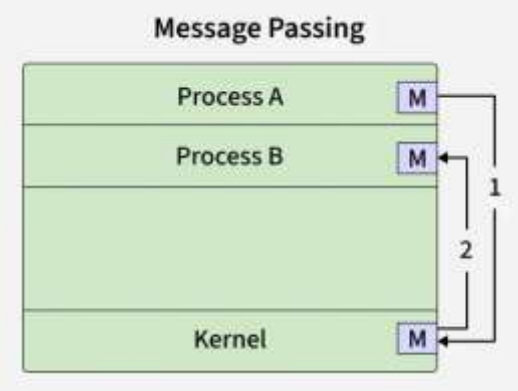


Ilustración 12. Mecanismo de paso de mensajes gestionado a través del núcleo (Kernel) del sistema operativo.

- **Tuberías (Pipes):** Una de las herramientas más potentes para el propósito de comunicar procesos es la tubería o *pipe*. Una tubería actúa como un canal lógico para el flujo de datos entre los procesos, y se define por su salida de un proceso y su entrada en otro proceso.

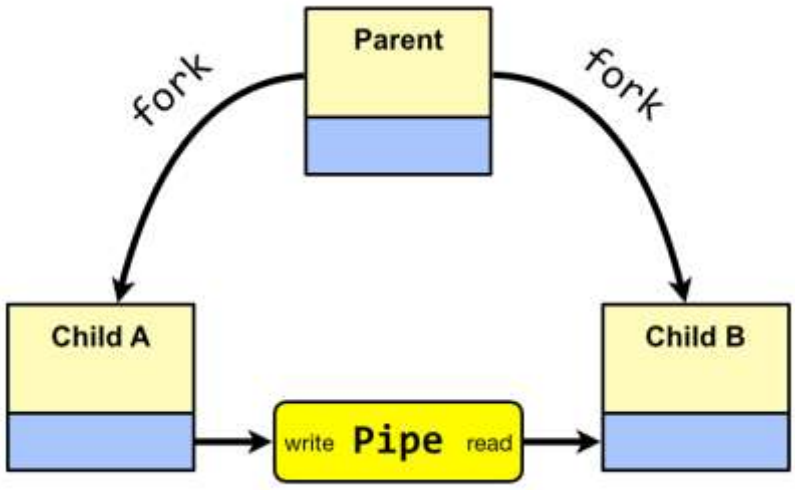


Ilustración 13. Comunicación entre procesos concurrentes mediante el uso de una tubería (pipe).

Tipos de interacción entre los procesos de la concurrencia

La interacción entre procesos concurrentes es la forma en que dos o más procesos pueden afectarse, relacionarse o trabajar entre sí mientras están en ejecución. Existen tres tipos principales:

- **Competencia entre procesos:** Sucede cuando dos o más procesos necesitan utilizar el mismo recurso, pero cada uno realiza su propia tarea y no necesita trabajar con los demás.
- **Cooperación por compartición:** Ocurre cuando los procesos utilizan un mismo recurso o información compartida para realizar sus actividades. Los procesos no necesariamente se comunican directamente pero sus acciones pueden afectar la información que utilizan los demás.
- **Cooperación por comunicación:** Ocurre cuando los procesos trabajan juntos y necesitan intercambiar información directamente para completar una actividad. Para esto pueden utilizar mecanismos de comunicación, como el envío y recepción de mensajes.

Ejemplos:

- **Competencia entre procesos:** Cuando dos programas diferentes pueden intentar utilizar una misma impresora, los programas no necesitan comunicarse entre ellos pero ambos necesitan el mismo recurso, por lo que el sistema operativo debe organizar el acceso.
- **Cooperación por compartición:** Cuando varios procesos pueden trabajar con una misma base de datos o una zona de memoria compartida, si uno modifica la información los demás podrían encontrarla modificada cuando la consulten.
- **Cooperación por comunicación:** Cuando un cliente envía una solicitud a un servidor, el cliente necesita comunicarse con el servidor para obtener una respuesta y continuar con su tarea.

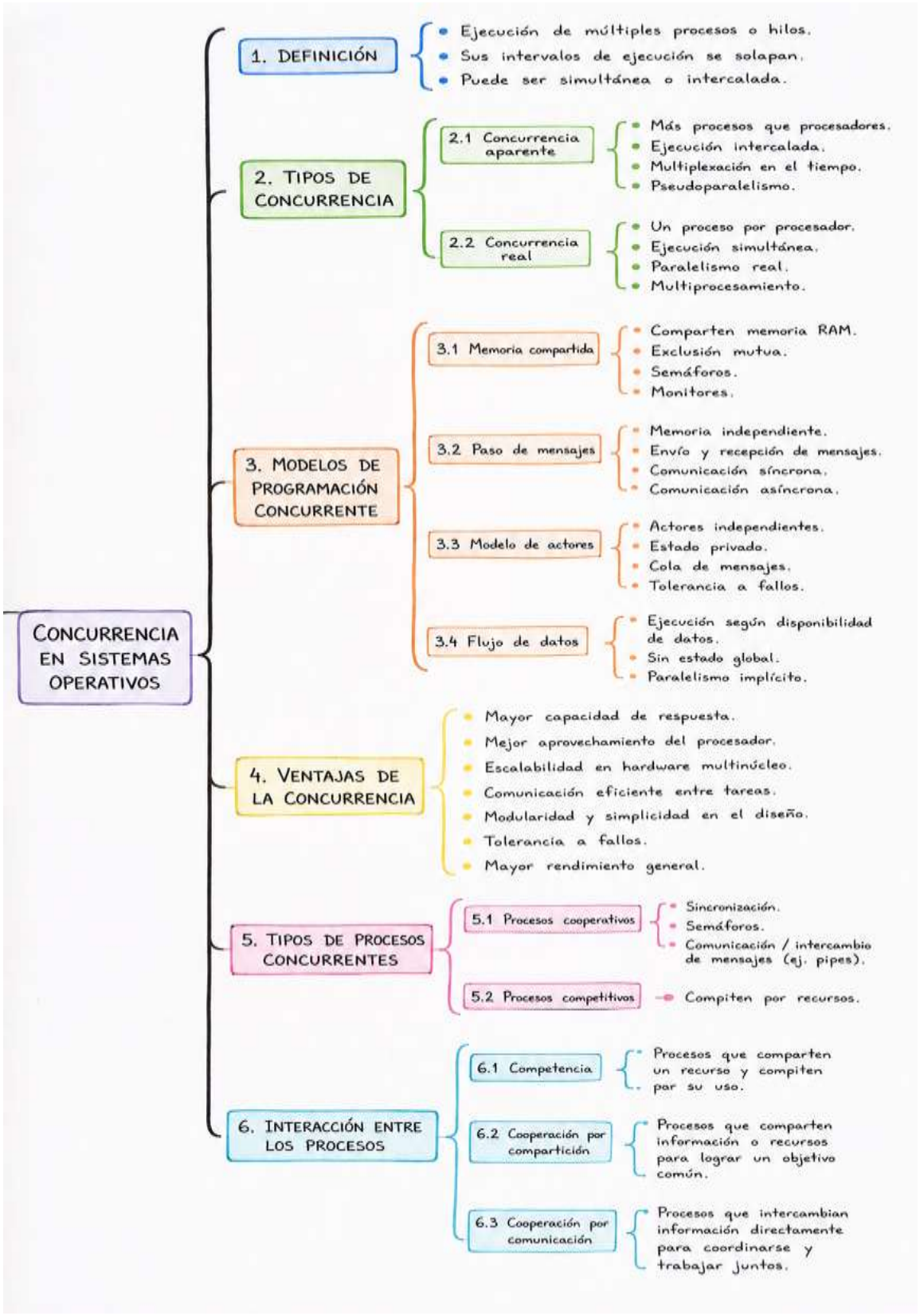






Ilustración 14. Ejemplos de tipos de interacción entre los procesos

Cuadro sinóptico



Conclusión General

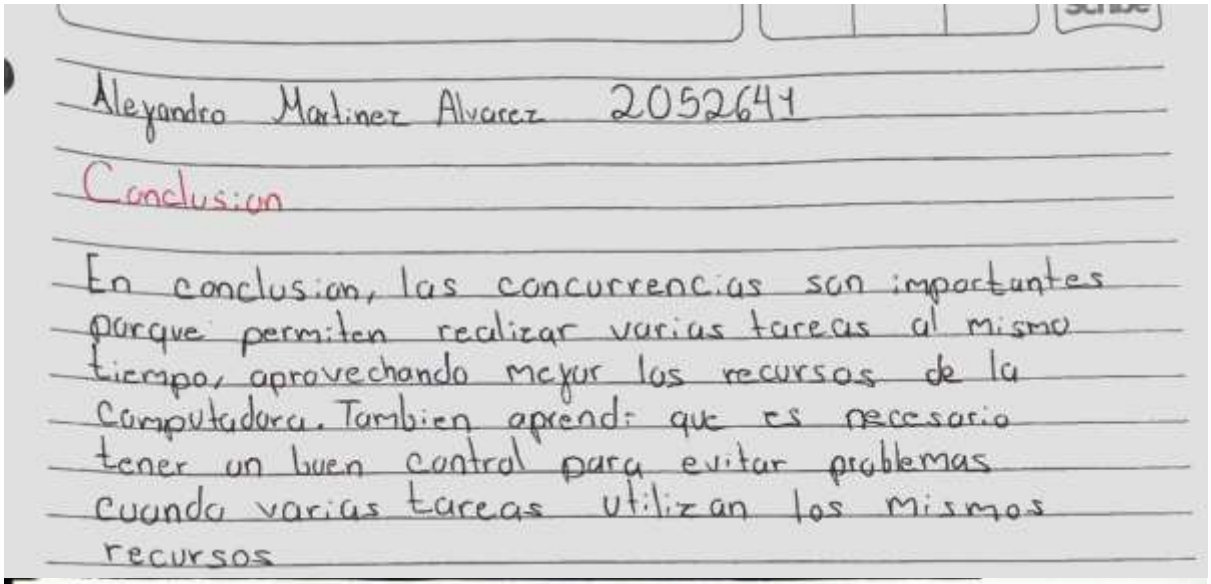
A lo largo de este reporte se analizó cómo los sistemas operativos gestionan la ejecución simultánea de procesos mediante los mecanismos de concurrencia. Se pudo comprender que la concurrencia no siempre implica una ejecución verdaderamente paralela: en sistemas con un solo procesador se presenta como concurrencia aparente, donde el sistema operativo alterna rápidamente entre procesos generando pseudoparalelismo, mientras que en sistemas multiprocesador es posible alcanzar una concurrencia real, con varios procesos ejecutándose al mismo tiempo en distintos núcleos.

Asimismo, se revisaron los principales modelos de programación concurrente memoria compartida, paso de mensajes, modelo de actores y flujo de datos cada uno con ventajas y desventajas propias según el tipo de hardware y la naturaleza del problema a resolver, desde aplicaciones locales de un solo equipo hasta sistemas distribuidos en la nube.

También se identificaron los beneficios que aporta la concurrencia a los sistemas operativos modernos, como una mayor capacidad de respuesta, un mejor aprovechamiento del procesador, escalabilidad en hardware multinúcleo y mayor tolerancia a fallos. Finalmente, se estudiaron los tipos de procesos concurrentes y las formas en que estos interactúan entre sí por competencia, por compartición o por comunicación, lo cual evidencia la importancia de contar con mecanismos de sincronización, como los semáforos y la exclusión mutua, para evitar condiciones de carrera y garantizar la integridad de la información.

En conjunto, estos conceptos demuestran que el control de la concurrencia es un pilar fundamental para el correcto funcionamiento de los sistemas operativos actuales, permitiendo administrar múltiples tareas de manera eficiente, segura y ordenada.

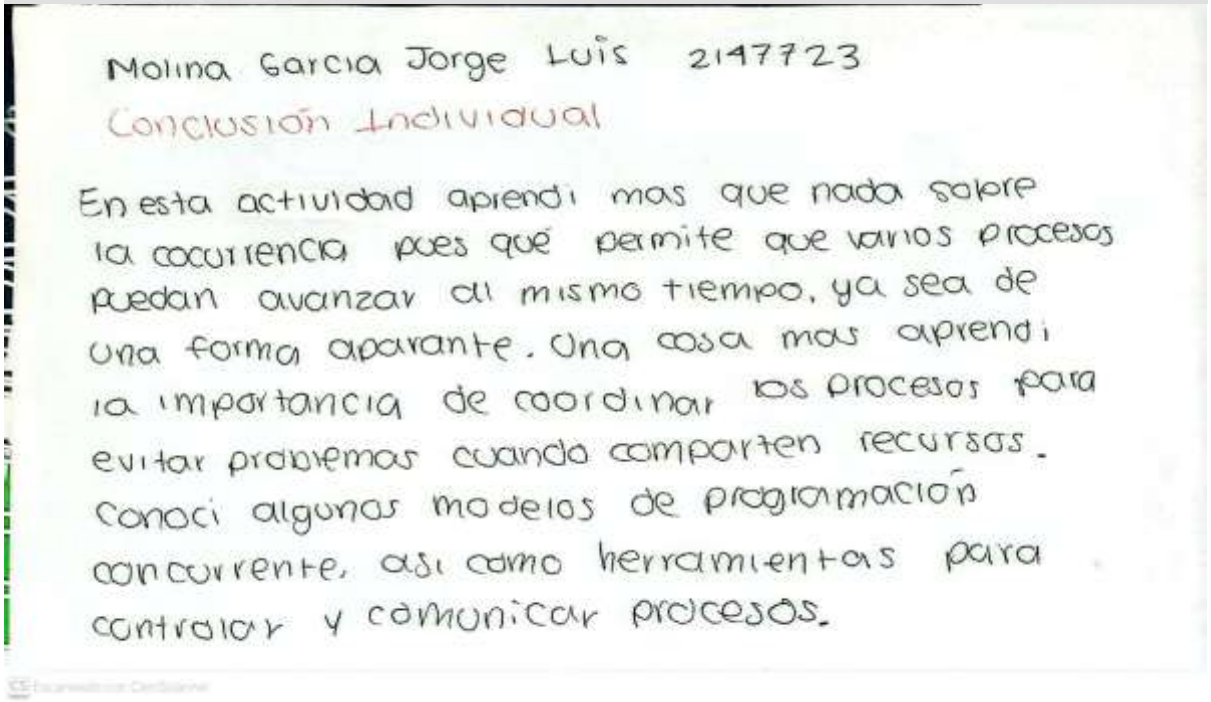
Conclusiones personales



Alejandro Martinez Alvarez 2052641

Conclusion

En conclusion, las concurrencias son importantes porque permiten realizar varias tareas al mismo tiempo, aprovechando mejor los recursos de la Computadora. Tambien aprendi que es necesario tener un buen control para evitar problemas cuando varias tareas utilizan los mismos recursos



Molina Garcia Jorge Luis 2147723

Conclusión Individual

En esta actividad aprendi mas que nada sobre la cocurrencia pues que permite que varios procesos puedan avanzar al mismo tiempo, ya sea de una forma aparante. Una cosa mas aprendi la importancia de coordinar los procesos para evitar problemas cuando comparten recursos. Conoci algunos modelos de programación concurrente, asi como herramientas para controlar y comunicar procesos.

Abner Daniel Chávez Romero 2173227 LAS

En conclusión, yo aprendí que la concurrencia es lo que permite que los sistemas operativos manejen múltiples tareas a la vez sin que la computadora colapse. Vimos que esto se logra intercalando procesos súper rápido o usando varios núcleos al mismo tiempo.

A día de hoy, la concurrencia es muy importante por el hecho que cada vez los programas son más exigentes y pesados.

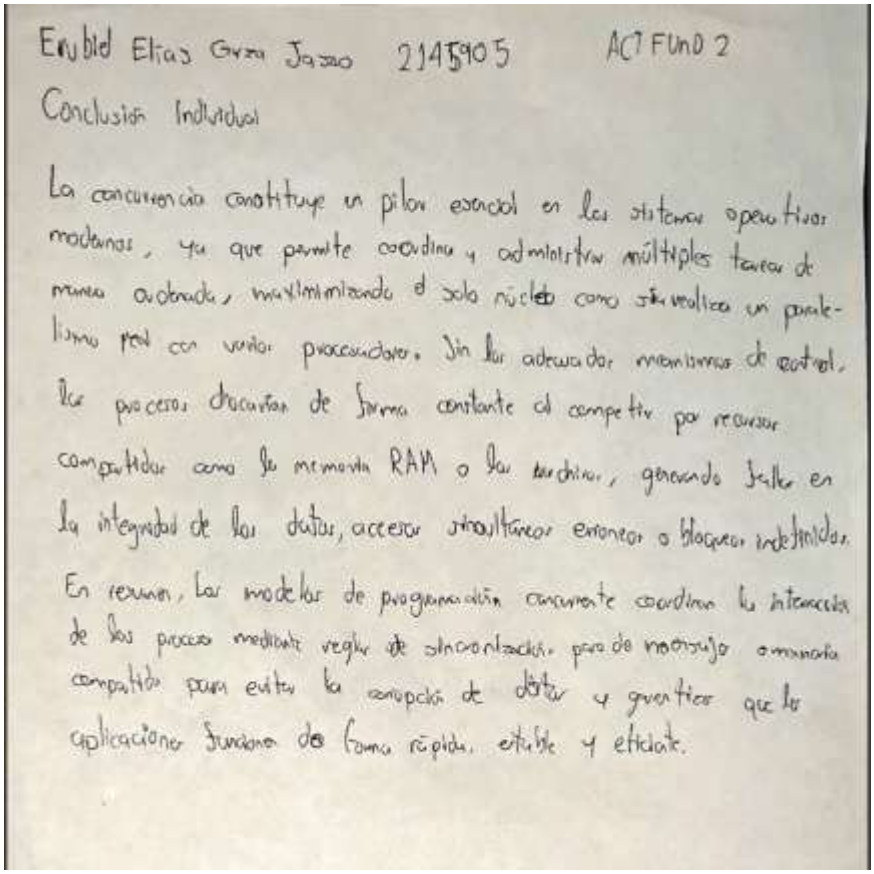
Flores Dávila Andrea Sarahy - 2149169 IAS

CONCLUSION INDIVIDUAL

La concurrencia es una caracteristica fundamental en los sistemas operativos, esta permite que varios procesos se puedan realizar de manera simultanea, investigamos sobre los tipos de concurrencia viendo cuando es que tienen una participación en los procedimientos.

La concurrencia permite mejorar el aprovechamiento de recursos en el sistema, dando una mayor capacidad para realizar varias tareas y dando eficiencia al momento de ejecutar los programas.

Es importante comprender el concepto de concurrencia para entender el control del sistema operativo en evitar conflictos en el sistema al ejecutar varias tareas.



Enabid Elias Cruz Jasso 2145905 ACT FUND 2

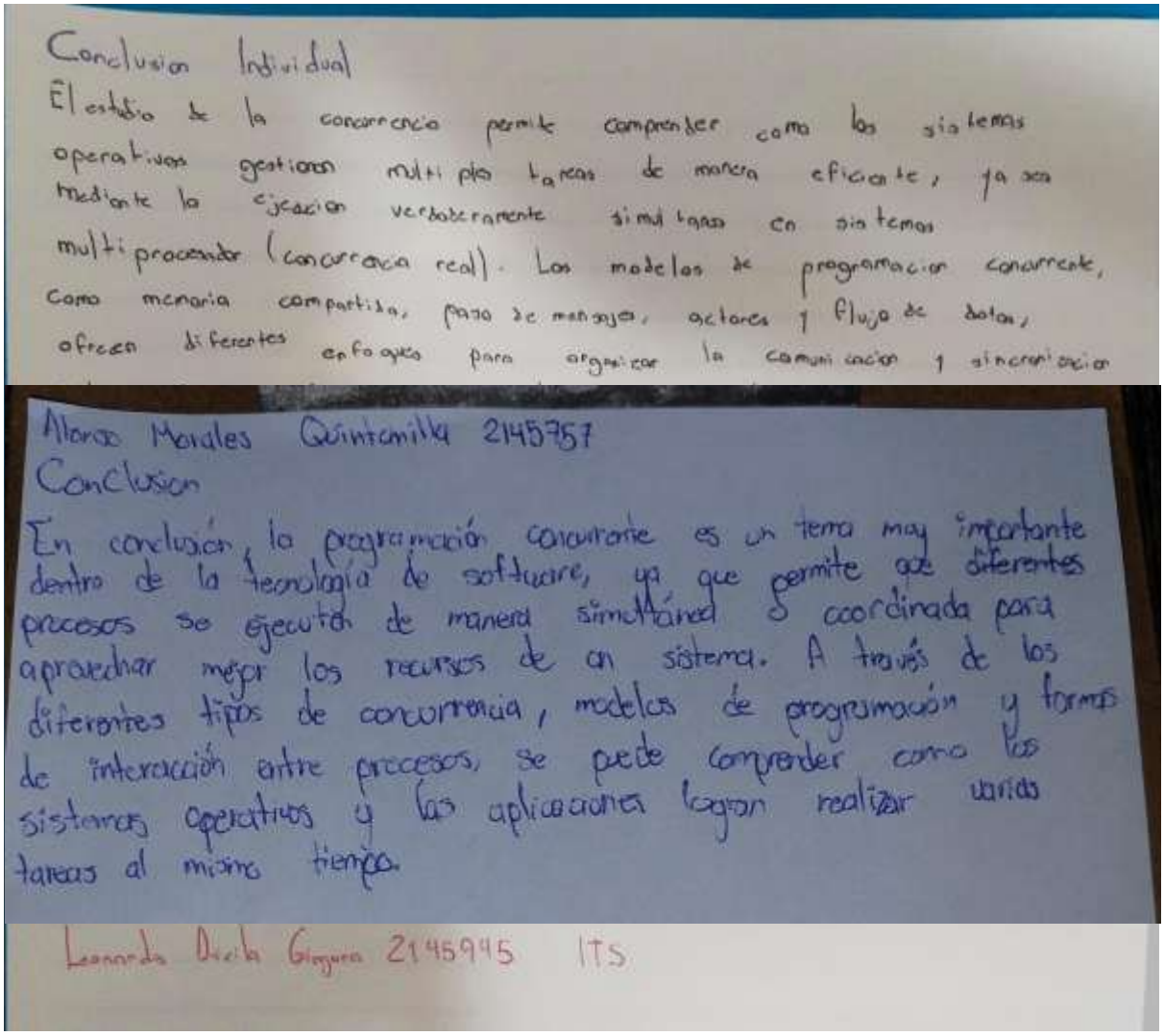
Conclusión Individual

La concurrencia constituye un pilar esencial en los sistemas operativos modernos, ya que permite coordinar y administrar múltiples tareas de manera ordenada, maximizando el solo núcleo como simultánea un paralelismo real con varios procesadores. Sin los adecuados mecanismos de control, los procesos chocarían de forma constante al competir por recursos compartidos como la memoria RAM o los archivos, generando fallos en la integridad de los datos, accesos simultáneos erróneos o bloqueos indefinidos.

En resumen, los modelos de programación concurrente coordinan la interacción de los procesos mediante reglas de sincronización, paso de mensajes o memoria compartida para evitar la corrupción de datos y garantizar que las aplicaciones funcionen de forma rápida, estable y eficiente.

Conclusion Individual – Actividad Fundamental 2
Manrroquín Tobías José Eduardo 2145922

Al realizar esta investigación sobre concurrencia, entendí que no se trata solo de un concepto técnico, si no de una herramienta clave para que los sistemas operativos aprovechen mejor sus recursos. Aprender sobre los tipos de concurrencia, los modelos de programación y las ventajas de aplicarla me permitió ver como la ejecución simultánea de procesos hace mas eficiente y flexible el trabajo de un sistema. Tambien me quedó claro que los distintos procesos concurrentes y sus interacciones requieren coordinación para evitar errores, lo cual refuerza la importancia de diseñar bien la comunicacion entre ellos. Siento que esta investigación me ayudó a valorar la concurrencia como un pilar fundamental en los sistemas operativos y a reconocer su impacto en la forma en que los usamos día a día.



Conclusion Individual

El estudio de la concurrencia permite comprender como los sistemas operativos gestionan multiples tareas de manera eficiente, ya sea mediante la ejecucion verdaderamente simultanea en sistemas multiprocesador (concurrencia real). Los modelos de programacion concurrente, como memoria compartida, paso de mensajes, actores y flujo de datos, ofrecen diferentes enfoques para organizar la comunicacion y sincronizacion

Alonso Morales Quintanilla 2145757

Conclusion

En conclusión, la programación concurrente es un tema muy importante dentro de la tecnología de software, ya que permite que diferentes procesos se ejecuten de manera simultánea y coordinada para aprovechar mejor los recursos de un sistema. A través de los diferentes tipos de concurrencia, modelos de programación y formas de interacción entre procesos, se puede comprender como los sistemas operativos y las aplicaciones logran realizar varias tareas al mismo tiempo.

Leonardo Davila Gregorio 2145945 ITS

Donovan Alexander Martínez García 2153829

Conclusión Individual

El estudio de la concurrencia nos permite entender cómo los sistemas operativos modernos logran ejecutar múltiples tareas de manera simultánea o alternado, maximizando el rendimiento del procesador y reduciendo tiempos de espera. En esta actividad analizamos desde su definición y los tipos de concurrencia existente, hasta los distintos modelos de programación que dictan la forma en que los los hilos e hilos de ejecución interactuan entre sí. Comprendí que aunque la ejecución concurrente ofrece grandes ventajas como la eficiencia y la modularidad también exige gestionar con cuidado los tipos de procesos e interacciones para evitar problemas complejos como bloqueos mutuos o condición de carrera. Dominar estos conceptos es importante para la comprensión del diseño interno de un sistema o

Conclusión individual Carolina Revuelta Martinez 2226953

Al terminar la investigación aprendí la importancia de la concurrencia en los sistemas operativos porque permite que diferentes procesos puedan avanzar al mismo tiempo. Estos procesos también pueden competir por recursos, comunicarse entre ellos, compartir información y organizarse. En general la investigación me ayudo a comprender cómo una computadora puede realizar varias tareas de manera eficiente y eficaz sin que unas interfieran con otras.

Edgar Daniel del Angel Hernández | 2094613 | 21 | 08 | 26

Conclusión:

En conclusión el control de concurrencia garantiza la coherencia y correcta sincronización de los datos cuando múltiples procesos o hilos se ejecutan de manera simultánea en un sistema. A través de su estudio, desde los mecanismos de bajo nivel y del sistema operativo. Hasta los modelos de diseño de software (Ocupo memoria compartida, paso de mensajes, flujo de datos, etc) se entiende que no hay una sola forma de solucionar algo, sino que cada problema se resuelve dependiendo de las necesidades específicas de arquitectura y rendimiento. Asimismo, estos modelos junto con patrones de comunicación como colas de mensajes o abstracciones asíncronas demuestra como la computación moderna logra solventar problemas como las condiciones de carrera, el interbloqueo y la inanición, dominar los bases de sincronización del sistema operativo resulta indispensable para el diseño de Software eficiente en entornos distribuidos y de procesamiento masivo.

Esdras Alexander Ruiz Treviño 2149382 IAS

Conclusion Personal

Este trabajo me ayudo a entender la diferencia entre concurrencia aparente y concurrencia real, asi como la importancia de mecanismos de sincronizacion, como los semaforos y la exclusion mutua para evitar errores entre procesos. Conocer los distintos modelos de programacion concurrente tambien me permitio comprender que su eleccion depende del tipo de sistema en el que se trabaje. En general, el reporte me dejo una base clara sobre como los sistemas operativos gestionan multiples tareas de forma eficiente y segura.

Conclusión Individual – Actividad Fundamental 2:
Karla Nozomi Salazar Arzola. 2144697. 1AS

Gracias a esta actividad comprendí que la concurrencia representa un principio fundamental en los sistemas operativos, sin darnos cuenta, están presentes en nuestra vida cotidiana. Usualmente se presenta en celulares haciendo varios movimientos a la vez como lo son enviar un mensaje mientras vemos en segundo plano una serie y se descarga un archivo.

Creo que este concepto no solo refleja algo teórico complejo, sino que ayuda a que la tecnología sea ágil, eficaz y eficiente.

Bibliografía

Carretero Pérez, J., García Carballeira, F., De Miguel Anasagasti, P., & Pérez Costoya, F. (2007). Sistemas operativos: una visión aplicada (2.ª ed.). McGraw-Hill.

Joyanes Aguilar, L. (2008). Fundamentos de programación: Algoritmos, estructura de datos y objetos (4.ª ed.). McGraw-Hill / Interamericana de España.

Stallings, W. (2005). Sistemas operativos: Aspectos internos y principios de diseño (5.ª ed.). Pearson Educación.

Universidad Carlos III de Madrid. (s. f.). Procesos concurrentes y problemas en la comunicación y la sincronización. Material docente de Sistemas Operativos.

Wolf, G., Ruiz, E., Bergero, F., & Meza, E. (2015). Fundamentos de sistemas operativos (1.ª ed.). Universidad Nacional Autónoma de México, Instituto de Investigaciones Económicas; Facultad de Ingeniería

Escoí, F. D. M., Villaplana, E. A., Minguet, A. R. E., Saiz, P. G., García-Fornes, A., de Juan Marín, R., & Roig, J. S. S. (2013). *Concurrencia y sistemas distribuidos*. Editorial de la Universidad Politécnica de Valencia.

Webgrafía

Ray, S. (2024, 4 de febrero). Concurrency in OS. Scaler Topics. https://www.scaler.com/topics/concurrency-in-os/

Ray, S. (2024, 4 de febrero). Concurrency in OS. Scaler Topics. https://www.scaler.com/topics/concurrency-in-os/

Linux Pills. (2015, 22 de abril). *Ejemplo de comunicacion cliente-servidor a través de memoria compartida*. https://linuxpills.wordpress.com/2015/04/22/ejemplo-de-comunicacion-cliente-servidor-a-traves-de-memoria-compartida/

Microsoft. (2026, 28 de abril). *Introducción a los flujos de datos y a la preparación de datos de autoservicio*. Microsoft Learn. https://learn.microsoft.com/es-es/power-bi/transform-model/dataflows/dataflows-introduction-self-service

Reiriz Cores, D. (2017, 5 de febrero). *Entendiendo la importancia del paso de mensajes*. Medium. https://medium.com/@diegoreico/entendiendo-el-paso-de-mensajes-ca859702457a

Tiwari, K. K. (2019, 28 de enero). *Actor Model in Nutshell*. Medium. https://medium.com/@KtheAgent/actor-model-in-nutshell-d13c0f81c8c7

Bibliografía de imágenes

Gamas Gutiérrez, R. (27 de febrero de 2013). *Definición y clasificación de sistemas multiprocesadores*. Sistemasumma.com. https://sistemasumma.com/2013/02/27/definicion-y-clasificacion-de-sistemas-multiprocesadores/

iStock. (s. f.). *Mujer joven emocionada escuchando música en auriculares inalámbricos a través de una* [Fotografía]. https://www.istockphoto.com/es/foto/mujer-joven-emocionada-escuchando-m%C3%BAsica-en-auriculares-inal%C3%A1mbricos-a-trav%C3%A9s-de-una-gm1958000977-557645082

Damiancipolat. (s. f.). GitHub - damiancipolat/NodeJS-Concurrencia-Paralelismo: Colección de ejemplos que muestran como aplicar concurrencia y paralelismo con nodejs. GitHub. https://github.com/damiancipolat/NodeJS-Concurrencia-Paralelismo

Glynis. (s. f.). *Sistemas operativos [Presentación PowerPoint]*. SlideServe. https://www.slideserve.com/glynis/sistemas-operativos-powerpoint-ppt-presentation

IT Release. (18 de julio de 2018). *Advantages and disadvantages of computer multitasking*. IT Release. https://itrelease.com/2018/07/advantages-and-disadvantages-of-computer-multitasking/